Proof Automation in Isabelle (Basic)

Course: Interactive Theorem Proving (7CCSACTP) — Lecture Notes Transcription

Context & Agenda

→ **Previous Videos:**

- Basic interface
- How to write terms / thms
- Proof tech's:
  - → Forward reasoning (FW)
  - → Backward reasoning (BW)
  - → Substitution (`subst`)
  - → Induction on numbers
- Lists:
  - → Recursion
  - → Proving using structural induction on lists

→ **This Video:**

Automation of Proofs in Isabelle
(Basic)

1. Simplification (`simp`)

- **Applies substitutions to a given goal.**
- **Hope:** It finishes the goal, or makes it easier for you.
- **Examples:**
  - length $(xs \mathbin{@} ys) = \text{length}\ xs + \text{length}\ ys$
  - $x > y \Longrightarrow (x - y) + c = (x + c) - y$
- It applies substitutions from **left to right**.
- **As many times as possible:**
  - Until no simplification rule can be substituted in the goal.
- **Does not always terminate!**

E.g. $f\,x = g\,x$, $g\,x = f\,x$ $\Longrightarrow$ **Loops forever!**

- Default **large library** of simp-rules:
  - `@`, `length`, etc.
  - Any recursive function f, f.simps are added automatically.
- **Manually extend** / remove simp-rules:

```
apply (simp add: rule1 del: rule2)
```

- **Note:** To remove *all* facts from simpset except f_1:

```
apply (simp only: f1)
```

2. The `auto` Proof Method

- `auto` ≈ **Simplification + Classical Reasoning**
  - → Simplification $\Longrightarrow$ repetitive substitution
  - → Classical reasoning $\Longrightarrow$ repetitive FW & BW reasoning
- Uses the **same simpset** as the simplifier.
- Has its own bank of **FW / BW reasoning lemmas**.
- **Recall:** To extend simpset for `simp`:

```
apply (simp add / del / only: rule1 rule2 ...)
```

- Same syntax works to modify simpset for `auto`.
- Add / remove a lemma to/from classical reasoning:
  - `apply (auto intro: fact1)` → Adds fact_1 to **BW**
  - `apply (auto dest: fact1)` → Adds fact_1 to **FW**
- **Recall:** FW reasoning is preferable for *manual* proofs.
- **For automation: BW reasoning is preferable**:
  - Helps better with debugging.
  - Usually more confident about the target conclusion.
  - Start from conclusion & find assumptions via BW reasoning.
- **Note:** `auto` **applies to all open goals**, unlike `simp`, `subst`, `rule`, etc.

3. Useful Notes & Attributes

- When proving a lemma[a] `lem1: "x + y = y + x"` to always use for simplification:

```
lem1 [simp]: "x + y = y + x"
```

- For Forward / Backward reasoning:

```
lem1 [dest / intro]: ...
```

- Predefined sets of simps for goal categories: E.g., `algebra_simps`, `field_simps`.
- **Naturals:** To translate between Suc(Suc n) and $n + 2$, use:

```
eval_nat_numerals
```

4. `force` & `fastforce`

- **More powerful** than `auto`.
- **Limitations:**
  - Cannot stop in the middle like `auto` or `simp`.
  - → **Not good for debugging!**
  - Unlike `auto`, they **only apply to the first goal!!!**
- Accepts options: `simp / intro / dest ...`